Reviewer Pack — Minimal Local Index in Algebraic Topology and Frege Proof De…

Entry d4f8f8d578d5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 23:51:09 UTC

Minimal Local Index in Algebraic Topology and Frege Proof Depth Correlation

Entry ID: `d4f8f8d578d5` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 23:51:09 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Local Systems) **Field B** (complexity object): Boolean Satisfiability (Frege Proof Complexity)

Statement:

> The local index of the fundamental group associated with a given CNF formula φ is linearly correlated with its Frege proof depth d(φ), such that log(2)^{local_index(φ)} = Θ(d(φ)).

Rationale (proposer's reasoning):

> Local systems in algebraic topology can capture information about connectivity, which might be related to the structure of logical proofs. A higher local index could indicate a more complex interaction between variables, potentially leading to deeper Frege proofs.

Taxonomy category: `topology_complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `3ac8d3027f89dc46`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for at least 80% of generated CNF formulas with n variables, the ratio of local index to Frege proof depth falls within a factor of 2^(local_index(φ)). The conjecture is falsified if any seed produces a ratio exceeding this factor.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 5 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"algebraic topology" AND "Frege proof complexity"` - `"CNF formula" AND local index AND Frege proof depth"` - `"fundamental group" AND CNF AND linear correlation`

Top relevant hits considered: - [http://arxiv.org/abs/0806.1148v2] Unsatisfiable CNF Formulas need many Conflicts - [http://arxiv.org/abs/2201.08895v4] On the Satisfaction Probabilities of k-CNF Formulas - [http://arxiv.org/abs/1006.3030v1] Satisfiability Thresholds for k-CNF Formula with Bounded Variable Intersections - [s2:10.1007/BF01294258] Proof complexity in algebraic systems and bounded depth Frege systems with modular counting - [s2:2510.17829] A Homological Separation of **P** from **NP** via Computational Topology and Category Theory

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(n * (n - 1)):
            clause = [random.randint(1, n), -random.randint(1, n)]
            if random.choice([True, False]):
                clause[0] *= -1
            if random.choice([True, False]):
                clause[1] *= -1
            clauses.append(clause)
        return clauses

    def frege_depth(cnf):
        def dpll(cnf, assignment):
            if not cnf:
                return 0
            unit_clauses = [c for c in cnf if len(c) == 1]
            if unit_clauses:
                literal, _ = unit_clauses[0]
                new_assignment = assignment.copy()
                new_assignment[abs(literal)] = literal > 0
                return 1 + dpll([c for c in cnf if literal not in c], new_assignment)
            pure_literals = {}
```

```
        for lit in set(abs(c) for c in cnf):
            pos_count, neg_count = sum(1 for c in cnf if lit in c), sum(1 for c in cnf if -lit
            if pos_count == 0:
                pure_literals[lit] = True
            elif neg_count == 0:
                pure_literals[lit] = False
        if not pure_literals:
            return float('inf')
        literal, value = next(iter(pure_literals.items()))
        new_assignment = assignment.copy()
        new_assignment[abs(literal)] = value
        return 1 + dpll([c for c in cnf if literal not in c], new_assignment)
    return dpll(cnf, {})

def local_index(cnf):
    n = len(set(abs(lit) for clause in cnf for lit in clause))
    # Simplified computation of local index
    return math.log(n, 2)

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    cnf = generate_cnf(n)
    li = local_index(cnf)
    fd = frege_depth(cnf)
    if fd == float('inf'):
        continue
    ratio = li / fd
    expected_ratio = math.log(2, 2) ** li
    results.append({
        "n": n,
        "local_index": li,
        "frege_depth": fd,
        "ratio": ratio,
        "expected_ratio": expected_ratio
    })

if not results:
    return {
        "metric_name": "Ratio",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "No valid instances generated"
    }

min_ratio = min(result["ratio"] for result in results)
max_ratio = max(result["ratio"] for result in results)
mean_ratio = sum(result["ratio"] for result in results) / len(results)
std_ratio = math.sqrt(sum((result["ratio"] - mean_ratio) ** 2 for result in results) / len(res

conjecture_holds = all(0.5 <= ratio / expected_ratio <= 2 for ratio, expected_ratio in zip(mir
counterexample = "" if conjecture_holds else "Ratio out of bounds"
```

```
    return {
        "metric_name": "Ratio",
        "metric_value": mean_ratio,
        "instances_tested": len(results),
        "n_max": max(result["n"] for result in results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if not all("metric_value" in r and r["metric_value"] is not None for r in results):
        print("RESULT: INCONCLUSIVE reason=missing_data")
    else:
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(re
        support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

        if support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_
        elif any(not r["conjecture_holds"] for r in results):
            first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
            print(f"RESULT: FALSIFIED counterexample='Ratio out of bounds' first_failing_seed={fir
        else:
            print("RESULT: INCONCLUSIVE reason=insufficient_support")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2a1b126d.py", line 114, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2a1b126d.py", line 68, in run_trial
    fd = frege_depth(cnf)
         ^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2a1b126d.py", line 55, in frege_depth
    return dpll(cnf, {})
           ^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2a1b126d.py", line 43, in dpll
    for lit in set(abs(c) for c in cnf):
               ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2a1b126d.py", line 43, in <genexpr>
```

```
    for lit in set(abs(c) for c in cnf):
                   ^^^^^^
TypeError: bad operand type for abs(): 'list'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Investigate and fix the error in the test code that caused it to crash. Once fixed, rerun the test to provide evidence for or against the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13982
2	propose	ollama_remote	glm4:latest	0	0	12425
3	preregistration	ollama_remote	glm4:latest	0	0	11476
4	novelty	ollama_remote	glm4:latest	0	0	8316
5	novelty	ollama_remote	glm4:latest	0	0	9453
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19291
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10566
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7815
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13699
10	judge	ollama_remote	glm4:latest	0	0	12549

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 119572 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/d4f8f8d578d5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d4f8f8d578d5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/d4f8f8d578d5.tar.gz` (if generated)